

Serie de manuales Java POO usando los CRUD de Artículos

Manuales progresivos por CRUD

Introducción general

Esta serie de manuales está pensada para acompañar una evolución progresiva de un proyecto Java de consola basado en un CRUD de artículos.

Cada proyecto representa una etapa distinta del aprendizaje.

La idea no es explicar todos los temas juntos desde el inicio, sino respetar la evolución exacta de los CRUDs:

1. CRUD Herencia Polimorfismo ToString.
2. CRUD Interfaces Calculable.
3. CRUD Categorías.
4. CRUD Generics y Repositorio.
5. CRUD Menús Separados.
6. CRUD Preentrega Completa.

Cada manual usa como base el código del CRUD correspondiente.

CRUD 1 — Herencia, Polimorfismo y ToString

Proyecto CRUD de artículos con herencia y polimorfismo

Tema central

Este proyecto introduce los conceptos fundamentales de Programación Orientada a Objetos en Java:

- clases
- objetos
- encapsulamiento
- herencia
- polimorfismo
- clases abstractas
- métodos abstractos
- sobrescritura de métodos
- uso de `toString()`
- uso de `instanceof`
- composición entre clases

En esta etapa todavía no se trabajan interfaces ni repositorios genéricos.

El objetivo es entender cómo un CRUD puede mejorar su diseño usando herencia y polimorfismo.

Clases principales del proyecto

El proyecto contiene estas clases:

```
src/com/techlab/articulo/App.java
src/com/techlab/articulo/model/Articulo.java
src/com/techlab/articulo/model/ArticuloElectronico.java
src/com/techlab/articulo/model/ArticuloAlimenticio.java
src/com/techlab/articulo/model/Categoria.java
```

La clase `App` contiene el menú principal y las operaciones CRUD.

La clase `Articulo` representa la idea general de artículo.

Las clases `ArticuloElectronico` y `ArticuloAlimenticio` representan tipos concretos de artículo.

La clase `Categoria` representa la categoría asociada a un artículo.

Qué es una clase

Antes de leer código, es importante entender una idea clave:

En Programación Orientada a Objetos, una clase funciona como un plano o una plantilla.

Por ejemplo, el plano de una casa define:

- cuántas habitaciones tendrá
- dónde estarán las puertas
- cómo será la estructura

Pero el plano NO es la casa real.

Con las clases pasa exactamente lo mismo.

La clase define cómo serán los objetos que se creen a partir de ella.

Una clase es un molde que define cómo serán los objetos.

En este proyecto, `Categoria` es una clase concreta:

```
public class Categoria {  
  
    private int codigo;  
    private String nombre;  
    private String descripcion;  
}
```

Esta clase indica que toda categoría tendrá:

- código
- nombre
- descripción

Pero la clase no es todavía una categoría real. Es solo la definición.

Qué es un objeto

Un objeto es una instancia concreta de una clase.

La palabra “instancia” suele confundir mucho al principio.

Cuando decimos:

“instanciar un objeto”

simplemente significa:

“crear un objeto real en memoria”

La clase define la estructura.

El objeto es el elemento real creado a partir de esa estructura.

Por ejemplo:

```
Categoria categoria = new Categoria(1, "Electrónica", "Productos  
tecnológicos");
```

Acá pasan varias cosas importantes:

Categoria categoria

Esta parte declara una variable.

La variable puede guardar referencias a objetos de tipo `Categoria`.

La palabra “referencia” significa que la variable apunta al objeto creado en memoria.

Luego:

```
new Categoria(...)
```

crea el objeto real.

Finalmente:

```
=
```

asigna la referencia del objeto a la variable `categoria`.

En `App`, cuando se precargan categorías, aparecen objetos reales:

```
categorias.add(new Categoria(1, "Electrónica", "Productos tecnológicos y electrónicos"));
categorias.add(new Categoria(2, "Periféricos", "Accesorios para computadora"));
categorias.add(new Categoria(3, "Alimentos", "Productos alimenticios"));
categorias.add(new Categoria(4, "Limpieza", "Artículos de limpieza del hogar"));
```

Cada `new Categoria(...)` crea un objeto distinto en memoria.

Encapsulamiento

El encapsulamiento es uno de los pilares más importantes de la Programación Orientada a Objetos.

La idea principal es:

```
“Los datos internos de un objeto no deberían modificarse libremente desde cualquier parte del sistema”.
```

¿Por qué?

Porque si cualquier clase pudiera cambiar cualquier dato sin control, el sistema sería muy difícil de mantener.

Aparecerían:

- errores difíciles de detectar
- datos inválidos
- comportamientos inesperados
- modificaciones accidentales

Por eso el encapsulamiento busca proteger el estado interno del objeto.

El encapsulamiento consiste en proteger los datos internos de una clase.

En Java se logra, principalmente, usando atributos privados y métodos públicos para acceder a ellos.

Ejemplo de `Articulo`:

```
private int codigo;  
private String nombre;  
private double precio;  
private Categoria categoria;
```

Estos atributos son `private`, por lo tanto no se pueden modificar directamente desde cualquier parte del programa.

No se debería hacer:

```
articulo.precio = -500;
```

En cambio, se usa un método setter:

```
articulo.setPrecio(nuevoPrecio);
```

Getters y setters

Los getters permiten leer atributos privados.

Los setters permiten modificar atributos privados.

Ejemplo:

```
public String getNombre() {  
    return nombre;  
}  
  
public void setNombre(String nombre) {
```

```
    this.nombre = nombre;
}
```

El método `getNombre()` devuelve el nombre del artículo.

El método `setNombre(...)` cambia el nombre del artículo.

Qué significa `this`

`this` representa al objeto actual.

Ejemplo:

```
public Categoria(int codigo, String nombre, String descripcion) {
    this.codigo = codigo;
    this.nombre = nombre;
    this.descripcion = descripcion;
}
```

La expresión:

```
this.codigo = codigo;
```

significa:

“Al atributo `codigo` de este objeto asigne el valor que llegó por parámetro”.

Herencia

La herencia es otro de los pilares fundamentales de la POO.

La idea principal es reutilizar comportamiento común.

En vez de repetir código muchas veces, se crea una clase base con todo lo compartido.

Después, otras clases más específicas heredan esa estructura.

La herencia permite que una clase reutilice atributos y métodos de otra clase.

En Java se usa `extends`.

En este proyecto:

```
public class ArtículoElectronico extends Artículo {
```

Y:

```
public class ArtículoAlimenticio extends Artículo {
```

Esto significa que:

- `ArtículoElectronico` es un `Artículo`
- `ArtículoAlimenticio` es un `Artículo`

`Artículo` es la clase padre.

`ArtículoElectronico` y `ArtículoAlimenticio` son clases hijas.

Por qué usar herencia en este CRUD

Un artículo electrónico tiene:

- código
- nombre
- precio
- categoría
- garantía en meses

Un artículo alimenticio tiene:

- código
- nombre
- precio
- categoría
- días para vencimiento

Los primeros cuatro atributos son comunes.

Si no hubiera herencia, habría que repetirlos en cada clase.

Con herencia, la clase `Artículo` concentra lo común:

```
public abstract class Artículo {  
  
    private int codigo;  
    private String nombre;  
    private double precio;  

```

```
private Categoria categoria;  
}
```

Y cada clase hija agrega lo específico.

`ArticuloElectronico` agrega:

```
private int garantiaMeses;
```

`ArticuloAlimenticio` agrega:

```
private int diasParaVencimiento;
```

Relación “es un”

La herencia debe usarse cuando la relación conceptual sea “es un”.

En este proyecto sí tiene sentido decir:

```
Un ArticuloElectronico es un Articulo.  
Un ArticuloAlimenticio es un Articulo.
```

Por eso se usa herencia.

En cambio, no tendría sentido decir:

```
Una Categoria es un Articulo.
```

Por eso `Categoria` no hereda de `Articulo`.

Composición

La composición aparece cuando una clase tiene un objeto de otra clase.

En `Articulo` existe este atributo:

```
private Categoria categoria;
```

Esto significa que un artículo tiene una categoría.

La relación no es “es un”.

La relación es “tiene un”.

Por eso corresponde composición y no herencia.

Clase abstracta

Una clase abstracta es una clase que no se puede instanciar directamente.

En el proyecto:

```
public abstract class Artículo {
```

Eso significa que no se puede hacer:

```
Articulo articulo = new Articulo(...);
```

No se puede crear un artículo genérico, porque `Articulo` está pensada como una clase base.

Por qué `Articulo` es abstracta

Porque en el sistema no queremos trabajar con artículos genéricos sin tipo.

Queremos artículos concretos:

- electrónicos
- alimenticios

`Articulo` representa lo común, pero no representa un objeto final que el usuario debería cargar.

Por eso se declara abstracta.

Una clase abstracta puede tener atributos

Sí.

`Articulo` es abstracta y tiene atributos:

```
private int codigo;  
private String nombre;
```

```
private double precio;  
private Categoria categoria;
```

Esto es válido.

Una clase abstracta puede tener estado.

Una clase abstracta puede tener constructor

Sí.

Aunque no se pueda hacer `new Artículo(...)`, la clase abstracta puede tener constructor.

Ese constructor se usa desde las clases hijas mediante `super(...)`.

`super`

`super` permite acceder a la clase padre.

En `ArticuloElectronico`:

```
public ArticuloElectronico(int codigo, String nombre, double precio,  
    Categoria categoria, int garantiaMeses) {  
    super(codigo, nombre, precio, categoria);  
    this.garantiaMeses = garantiaMeses;  
}
```

La línea:

```
super(codigo, nombre, precio, categoria);
```

llama al constructor de `Articulo` para inicializar los atributos comunes.

Después se inicializa lo propio del electrónico:

```
this.garantiaMeses = garantiaMeses;
```

Métodos abstractos

Un método abstracto es un método sin implementación.

Solo declara que ese comportamiento debe existir.

En `Articulo` aparecen métodos como:

```
public abstract String getTipoArticulo();
```

Y:

```
protected abstract String getDetalleEspecifico();
```

La clase padre no sabe cómo responder eso porque depende del subtipo.

Un electrónico debe responder una cosa.

Un alimenticio debe responder otra.

Implementación en las clases hijas

En `ArticuloElectronico`:

```
@Override
public String getTipoArticulo() {
    return "Electrónico";
}

@Override
protected String getDetalleEspecifico() {
    return "garantía=" + garantiaMeses + " meses";
}
```

En `ArticuloAlimenticio`:

```
@Override
public String getTipoArticulo() {
    return "Alimenticio";
}

@Override
protected String getDetalleEspecifico() {
    return "días para vencimiento=" + diasParaVencimiento;
}
```

Cada clase hija está obligada a implementar esos métodos porque hereda de una clase abstracta que los declaró.

Es obligatorio que una clase abstracta tenga métodos abstractos

No.

Una clase puede ser abstracta y no tener ningún método abstracto.

Ejemplo válido:

```
public abstract class Artículo {  
  
    private int codigo;  
  
    public int getCodigo() {  
        return codigo;  
    }  
}
```

La clase sigue siendo abstracta porque no se puede instanciar.

A veces se hace una clase abstracta solo para impedir que se creen objetos de esa clase base.

Una clase normal puede tener métodos abstractos

No.

Esto es incorrecto:

```
public class Artículo {  
    public abstract String getTipoArticulo();  
}
```

Si una clase tiene un método abstracto, la clase obligatoriamente debe ser abstracta.

Correcto:

```
public abstract class Artículo {  
    public abstract String getTipoArticulo();  
}
```

Se puede tener herencia sin clases abstractas

Sí.

La herencia no depende obligatoriamente de las clases abstractas.

Podríamos tener:

```
public class Artículo {  
}  
  
public class ArtículoElectronico extends Artículo {  
}
```

Eso es herencia válida.

La diferencia es que si `Artículo` no fuera abstracta, se podrían crear artículos genéricos.

En este proyecto no queremos eso.

Por eso `Artículo` es abstracta.

Polimorfismo

El polimorfismo suele ser uno de los conceptos más difíciles al comenzar POO.

Por eso es importante entender primero la idea conceptual.

La palabra:

polimorfismo

significa:

“muchas formas”

En Java, el polimorfismo permite que una variable de tipo padre pueda apuntar a objetos de distintas clases hijas.

Esto hace que el código sea mucho más flexible.

Polimorfismo significa “muchas formas”.

En Java, permite tratar objetos de clases hijas como si fueran objetos de la clase padre.

En el CRUD se usa una lista común:

```
ArrayList<Articulo> articulos = new ArrayList<>();
```

Esa lista puede guardar:

```
new ArticuloElectronico(...)
```

Y también:

```
new ArticuloAlimenticio(...)
```

Porque ambos son artículos.

Polimorfismo en el alta de artículos

En el método de ingreso de artículos, se puede declarar:

```
Articulo articulo;
```

Luego, según la opción elegida por el usuario, esa variable puede apuntar a un electrónico:

```
articulo = new ArticuloElectronico(codigo, nombre, precio, categoria,  
garantiaMeses);
```

O a un alimenticio:

```
articulo = new ArticuloAlimenticio(codigo, nombre, precio, categoria,  
diasParaVencimiento);
```

La variable es de tipo `Articulo`, pero el objeto real puede ser de distintos tipos.

Polimorfismo en el listado

Cuando el programa lista artículos, puede hacer:

```
for (Articulo articulo : articulos) {  
    System.out.println(articulo);  
}
```

El `for` no necesita saber si cada artículo es electrónico o alimenticio.

Java ejecuta el comportamiento correspondiente según el objeto real.

toString()

toString() sirve para representar un objeto como texto.

Todas las clases heredan toString() desde Object, pero la implementación original no suele ser útil para mostrar datos al usuario.

Por eso se sobrescribe.

En Categoría:

```
@Override
public String toString() {
    return "Categoría {" +
        "código=" + codigo +
        ", nombre='" + nombre + '\'' +
        ", descripción='" + descripcion + '\'' +
        '}';
}
```

Así, cuando se imprime una categoría, se ve información entendible.

toString() en Artículo

La clase Artículo tiene un toString() común para todos los artículos.

Ese método muestra:

- código
- nombre
- precio
- categoría
- tipo
- detalle específico

Lo interesante es que usa métodos abstractos:

```
getTipoArticulo()
getDetalleEspecifico()
```

Eso permite que el toString() sea común, pero que parte del resultado dependa del subtipo real.

`super.toString()`

Las clases hijas pueden reutilizar el `toString()` de la clase padre.

Ejemplo:

```
@Override
public String toString() {
    return super.toString() + " [subtipo electrónico]";
}
```

`super.toString()` llama al método `toString()` de `Articulo`.

Luego la clase hija agrega información propia.

`instanceof`

`instanceof` permite preguntar si un objeto pertenece a una clase concreta.

En el CRUD se usa para modificar campos específicos.

Ejemplo:

```
if (articulo instanceof ArticuloElectronico) {
    ArticuloElectronico electronico = (ArticuloElectronico) articulo;
    electronico.setGarantiaMeses(nuevaGarantia);
}
```

Esto es necesario porque la variable está declarada como `Articulo`, pero `setGarantiaMeses(...)` solo existe en `ArticuloElectronico`.

Primero se valida el tipo real con `instanceof`.

Después se hace el casteo.

Ideas clave del CRUD 1

- Una clase define estructura y comportamiento.
- Un objeto es una instancia real creada a partir de una clase.
- El encapsulamiento protege atributos internos.
- La herencia reutiliza comportamiento común.
- La composición representa relaciones “tiene un”.
- Una clase abstracta no puede instanciarse.
- Un método abstracto obliga a las hijas a implementar comportamiento.

- El polimorfismo permite tratar distintos subtipos usando un tipo común.
- `toString()` permite mostrar objetos de forma legible.
- `instanceof` permite verificar el tipo real de un objeto.

Errores comunes de alumnos principiantes

Pensar que una clase y un objeto son lo mismo

La clase es el molde.

El objeto es la instancia creada.

Pensar que `private` significa “inútil”

Muchos alumnos creen que si un atributo es privado no puede usarse.

Sí puede usarse.

Simplemente debe accederse mediante métodos públicos.

Confundir herencia con composición

Si la relación no es “es un”, probablemente no corresponda herencia.

Pensar que polimorfismo significa solo sobrescribir métodos

La sobrescritura es parte del polimorfismo, pero el concepto importante es:

una referencia padre puede apuntar a distintos subtipos

Pensar que una clase abstracta está “vacía”

Una clase abstracta puede tener:

- atributos
- constructores
- métodos normales
- lógica implementada

Lo único que no puede hacerse es crear objetos directamente de ella.

Cierre del CRUD 1

Este primer proyecto permite entender la base de la POO en Java.

Los conceptos más importantes son:

- una clase define estructura
- un objeto es una instancia real
- el encapsulamiento protege atributos
- la herencia reutiliza código común
- una clase abstracta no puede instanciarse
- un método abstracto obliga a las hijas a implementar comportamiento
- el polimorfismo permite trabajar con distintos subtipos usando un tipo común
- `toString()` permite mostrar objetos de forma legible

CRUD 2 — Interfaz Calculable

Proyecto CRUD de artículos con interfaz Calculable

Tema central

Este proyecto parte del CRUD anterior y agrega un concepto nuevo: las interfaces.

El nuevo archivo incorporado es:

```
src/com/techlab/articulo/interfaces/Calculable.java
```

Además, la clase `Articulo` ahora implementa esa interfaz:

```
public abstract class Articulo implements Calculable {
```

El objetivo de esta clase es entender cómo una interfaz puede definir un comportamiento común sin imponer una implementación única.

Qué problema resuelve esta clase

En la clase anterior, todos los artículos tenían precio.

Pero ahora aparece una nueva necesidad: calcular el precio final.

No todos los artículos calculan el precio final igual.

Un electrónico puede tener recargo si tiene mucha garantía.

Un alimenticio puede tener descuento si está cerca de vencer.

Entonces necesitamos un método común:

```
calcularPrecioFinal()
```

Pero con implementación distinta según el tipo de artículo.

Qué es una interfaz

Una interfaz en Java define un contrato.

Un contrato dice qué métodos debe tener una clase, pero no necesariamente cómo se resuelven.

En el proyecto:

```
public interface Calculable {  
    double calcularPrecioFinal();  
}
```

Esto significa:

“Todo lo que sea `Calculable` debe tener un método `calcularPrecioFinal()` que devuelva un `double`”.

Por qué se llama `Calculable`

El nombre representa una capacidad.

Algo calculable es algo que puede realizar un cálculo.

En este caso, los artículos pueden calcular su precio final.

La interfaz no dice si el objeto es un artículo, una factura o un servicio.

Solo dice que ese objeto sabe calcular algo.

`implements`

Para implementar una interfaz se usa la palabra `implements`.

En el proyecto:

```
public abstract class Articulo implements Calculable {
```

Esto significa que todo artículo será calculable.

Como `Articulo` es abstracta, puede no implementar directamente `calcularPrecioFinal()` y dejar que las clases hijas lo hagan.

Implementación en `ArticuloElectronico`

`ArticuloElectronico` implementa el cálculo así:

```
@Override
public double calcularPrecioFinal() {
    if (garantiaMeses > 12) {
        return precio * 1.10;
    }
    return precio;
}
```

La regla didáctica es:

- si la garantía supera los 12 meses, se aplica 10% de recargo
 - si no, el precio final es igual al precio base
-

Implementación en `ArticuloAlimenticio`

`ArticuloAlimenticio` implementa otra lógica:

```
@Override
public double calcularPrecioFinal() {
    if (diasParaVencimiento <= 3) {
        return precio * 0.80;
    }

    if (diasParaVencimiento <= 7) {
        return precio * 0.90;
    }

    return precio;
}
```

La regla didáctica es:

- si vence en 3 días o menos, se aplica 20% de descuento
 - si vence en 7 días o menos, se aplica 10% de descuento
 - si no, conserva el precio base
-

Relación con polimorfismo

Ahora el polimorfismo no solo aplica a los métodos abstractos de `Articulo`, sino también al método de la interfaz.

Cuando el programa imprime un artículo, el `toString()` de `Articulo` puede llamar a:

```
calcularPrecioFinal()
```

Pero la implementación real depende del subtipo.

Si el objeto es electrónico, se ejecuta la lógica de garantía.

Si el objeto es alimenticio, se ejecuta la lógica de vencimiento.

Clase abstracta vs interfaz

`Articulo` como clase abstracta representa una relación de identidad:

```
ArticuloElectronico es un Articulo.  
ArticuloAlimenticio es un Articulo.
```

`Calculable` como interfaz representa una capacidad:

```
Un artículo puede calcular su precio final.
```

La clase abstracta modela qué es el objeto.

La interfaz modela qué puede hacer el objeto.

Una clase puede heredar e implementar a la vez

En Java, una clase puede heredar de una sola clase, pero puede implementar interfaces.

Ejemplo de esta etapa:

```
public abstract class Articulo implements Calculable {
```

Y las hijas siguen heredando de `Articulo`:

```
public class ArtículoElectronico extends Artículo {
```

Entonces la estructura completa queda así:

```
Calculable
  ↑
Artículo
  ↑
ArtículoElectronico
ArtículoAlimenticio
```

Métodos implementados en interfaces

En versiones modernas de Java, las interfaces pueden tener métodos implementados.

Esto existe desde Java 8 mediante métodos `default` y `static`.

En Java 14 también se pueden usar.

Ejemplo didáctico:

```
public interface Calculable {

    double calcularPrecioFinal();

    default String descripcionCalculo() {
        return "Este objeto puede calcular un valor final";
    }
}
```

El método abstracto es:

```
calcularPrecioFinal();
```

El método implementado es:

```
descripcionCalculo()
```

Las clases están obligadas a implementar el método abstracto, pero no están obligadas a reescribir el método `default`.

Cierre del Manual 2

Este proyecto agrega interfaces para separar mejor las responsabilidades.

Los conceptos clave son:

- una interfaz define un contrato
 - `implements` indica que una clase cumple ese contrato
 - una interfaz modela capacidades
 - una clase abstracta modela una base común
 - distintas clases pueden implementar el mismo método de formas diferentes
 - desde Java 8 existen métodos implementados en interfaces mediante `default` y `static`
 - en Java 14 esto también está disponible
-

CRUD 3 — Categorías

Proyecto CRUD de categorías y artículos relacionados

Tema central

Este proyecto agrega un CRUD completo para categorías.

Antes, las categorías estaban precargadas.

Ahora el usuario puede:

- ingresar categorías
- listar categorías
- consultar una categoría
- modificar una categoría
- eliminar una categoría

La clase `App` crece porque ahora administra dos recursos:

- artículos
 - categorías
-

Nuevas responsabilidades del sistema

En las clases anteriores, `Categoria` existía como modelo, pero no tenía CRUD completo.

En esta clase, `Categoria` pasa a ser una entidad administrada por el usuario.

Esto permite trabajar una idea muy importante:

Un sistema puede tener varios CRUD relacionados entre sí.

En este caso:

- el CRUD de categorías administra categorías
- el CRUD de artículos necesita categorías existentes para asociarlas a los artículos

Métodos nuevos en `App`

Aparecen métodos como:

```
ingresarCategoria(...)  
listarCategorias(...)  
consultarCategoria(...)  
modificarCategoria(...)  
eliminarCategoria(...)
```

También aparecen validaciones relacionadas:

```
existeCategoriaPorNombre(...)  
categoriaTieneArticulosAsociados(...)
```

CRUD de categorías

CRUD significa:

- Create: crear
- Read: leer/listar/consultar
- Update: modificar
- Delete: eliminar

En esta clase, ese patrón se aplica a `Categoria`.

Alta de categoría

El alta crea un nuevo objeto `Categoria` y lo agrega a la lista.

Conceptualmente hace esto:


```
Categoria categoria = new Categoria(codigo, nombre, descripcion);
categorias.add(categoria);
```

El usuario ingresa los datos.

El programa valida que sean correctos.

Luego se crea el objeto.

Listado de categorías

El listado recorre la colección de categorías:

```
for (Categoria categoria : categorias) {
    System.out.println(categoria);
}
```

Gracias al `toString()` de `Categoria`, cada objeto se muestra de forma legible.

Consulta de categoría

La consulta busca una categoría por código.

Conceptualmente:

```
Categoria categoria = buscarCategoriaPorCodigo(codigo);
```

Si la encuentra, la muestra.

Si no la encuentra, informa que no existe.

Modificación de categoría

Modificar una categoría implica:

1. pedir el código
2. buscar la categoría
3. pedir nuevos datos
4. actualizar el objeto usando setters

Ejemplo:

```
categoria.setNombre(nuevoNombre);  
categoria.setDescripcion(nuevaDescripcion);
```

Esto refuerza el uso de encapsulamiento.

Los atributos no se modifican directamente.

Se modifican mediante métodos públicos.

Eliminación de categoría

Eliminar una categoría no es tan simple como eliminar un artículo.

¿Por qué?

Porque puede haber artículos asociados a esa categoría.

Si se elimina una categoría usada por artículos, esos artículos quedarían apuntando a una categoría inexistente.

Por eso aparece una validación importante:

```
categoriaTieneArticulosAsociados(...)
```

Integridad de datos

La integridad de datos significa que la información del sistema debe mantenerse coherente.

En este CRUD, una regla de integridad sería:

```
No eliminar una categoría si existen artículos asociados a ella.
```

Esto evita inconsistencias.

Por ejemplo, si existe un artículo "Mouse" asociado a la categoría "Periféricos", no conviene eliminar "Periféricos" sin resolver antes qué pasará con ese artículo.

Relación entre artículo y categoría

La clase `Articulo` tiene:

```
private Categoria categoria;
```

Esto significa que un artículo tiene asociada una categoría completa, no solo un texto.

Esto permite acceder a datos de la categoría desde el artículo:

```
categoria.getNombre()
```

Por eso en el `toString()` de artículo se puede mostrar el nombre de la categoría.

Validación de nombres duplicados

Aparece la necesidad de evitar categorías repetidas.

Para eso se usa una validación conceptual como:

```
existeCategoriaPorNombre(nombre)
```

Esto evita que el usuario cargue dos categorías con el mismo nombre.

Aunque técnicamente podrían tener distinto código, funcionalmente sería confuso.

Mayor complejidad en `App`

En esta etapa, `App` empieza a concentrar demasiadas responsabilidades.

Maneja:

- menú principal
- CRUD de artículos
- CRUD de categorías
- validaciones
- búsquedas
- lectura de datos por consola

Esto funciona para aprender, pero empieza a mostrar una necesidad de mejora: separar responsabilidades.

Esa mejora aparecerá en proyectos posteriores.

Cierre del Manual 3

Este proyecto enseña que un CRUD puede crecer y administrar varias entidades relacionadas.

Los conceptos clave son:

- CRUD de una segunda entidad
- relación entre objetos
- composición entre `Articulo` y `Categoria`
- validaciones de negocio
- integridad de datos
- uso de setters para modificar objetos
- necesidad de empezar a separar responsabilidades cuando una clase crece demasiado

CRUD 4 — Generics y Repositorio

Proyecto CRUD con repositorio genérico

Tema central

Este proyecto incorpora dos conceptos muy importantes:

- generics
- repositorio genérico

Aparecen nuevos archivos:

```
src/com/techlab/articulo/interfaces/Identificable.java  
src/com/techlab/articulo/repository/Repositorio.java
```

El objetivo es evitar repetir lógica para administrar listas de distintos tipos de objetos.

Problema antes de usar repositorio

Antes, la clase `App` tenía listas y métodos de búsqueda para cada entidad.

Por ejemplo:

```
ArrayList<Articulo> articulos  
ArrayList<Categoria> categorias
```

Y también métodos como:

```
buscarArticuloPorCodigo(...)  
buscarCategoriaPorCodigo(...)
```

Ambos métodos hacen algo muy parecido: buscar un objeto por código.

Entonces aparece una oportunidad de generalizar.

Qué es un generic

Los generics suelen ser uno de los temas que más cuestan al principio.

La razón es que introducen una idea nueva:

```
trabajar con tipos flexibles
```

Hasta ahora veníamos creando clases concretas.

Por ejemplo:

```
ArrayList<Articulo>
```

O:

```
ArrayList<Categoria>
```

Pero aparece un problema:

muchas veces queremos reutilizar lógica sin importar exactamente el tipo.

Por ejemplo, buscar por código funciona igual para:

- artículos
- categorías
- usuarios
- pedidos

Entonces surge la idea de crear clases genéricas.

Un generic permite crear clases o métodos que trabajan con un tipo flexible.

En vez de crear un repositorio solo para artículos y otro solo para categorías, se crea uno genérico.

```
public class Repositorio<T extends Identificable> {
```

La letra **T** representa un tipo.

Puede ser:

- **Articulo**
- **Categoria**
- otro tipo que cumpla la condición

Qué significa **T**

Esta parte suele generar mucha confusión al principio:

```
Repositorio<T>
```

Muchos alumnos piensan que **T** es una clase real.

Pero no lo es.

T es simplemente una letra elegida por convención.

Podría haberse llamado:

```
Repositorio<X>  
Repositorio<Dato>  
Repositorio<Tipo>
```

La letra **T** normalmente viene de:

```
Type
```

que significa "tipo".

T es un parámetro de tipo.

No es una clase concreta.

Es una forma de decir:

"Este repositorio va a trabajar con algún tipo, pero todavía no sé cuál".

Cuando se crea el repositorio, se define el tipo real:

```
Repositorio<Articulo> repositorioArticulos = new Repositorio<>();  
Repositorio<Categoria> repositorioCategorias = new Repositorio<>();
```

El mismo código sirve para dos tipos distintos.

Qué significa `T extends Identificable`

La clase se declara así:

```
public class Repositorio<T extends Identificable> {
```

Esto significa:

“T puede ser cualquier tipo, siempre que implemente `Identificable`”.

¿Por qué hace falta esta restricción?

Porque el repositorio necesita llamar a:

```
objeto.getCodigo()
```

Si Java no supiera que `T` tiene `getCodigo()`, no permitiría compilar el código.

Interfaz `Identificable`

La interfaz es:

```
public interface Identificable {  
  
    int getCodigo();  
}
```

Esto obliga a que las clases identificables tengan un método `getCodigo()`.

En este proyecto:

```
public abstract class Articulo implements Calculable, Identificable
```

Y:

```
public class Categoria implements Identificable
```

Entonces tanto artículos como categorías pueden ser guardados en el repositorio genérico.

Repositorio genérico

El repositorio tiene una lista interna:

```
private final ArrayList<T> lista;
```

Como la lista es de tipo `T`, puede guardar objetos del tipo definido al crear el repositorio.

Si se crea:

```
Repositorio<Categoria> repositorioCategorias = new Repositorio<>();
```

Entonces internamente trabaja con categorías.

Si se crea:

```
Repositorio<Articulo> repositorioArticulos = new Repositorio<>();
```

Entonces internamente trabaja con artículos.

Método `agregar`

```
public boolean agregar(T objeto) {  
    if (objeto == null) {  
        return false;  
    }  
  
    if (buscarPorCodigo(objeto.getCodigo()) != null) {  
        return false;  
    }  
  
    return lista.add(objeto);  
}
```

Este método sirve para cualquier `T` identificable.

Puede agregar una categoría.

Puede agregar un artículo.

La validación de duplicado funciona porque todos tienen código.

Método `listar`

```
public List<T> listar() {  
    return new ArrayList<>(lista);  
}
```

Devuelve una copia de la lista.

Esto es importante para proteger la lista interna.

Si devolviera directamente la lista original, cualquier parte externa del programa podría modificarla sin pasar por el repositorio.

Método `buscarPorCodigo`

```
public T buscarPorCodigo(int codigo) {  
    for (T objeto : lista) {  
        if (objeto.getCodigo() == codigo) {  
            return objeto;  
        }  
    }  
    return null;  
}
```

Este método devuelve un objeto de tipo `T`.

Si el repositorio es de categorías, devuelve una `Categoria`.

Si el repositorio es de artículos, devuelve un `Articulo`.

Método `eliminar`

```
public boolean eliminar(T objeto) {  
    if (objeto == null) {  
        return false;  
    }  
}
```

```
        return lista.remove(objeto);  
    }
```

Elimina el objeto recibido.

También valida que no sea `null`.

Beneficio principal del repositorio

Antes se necesitaba lógica repetida para artículos y categorías.

Ahora se centraliza en una sola clase:

```
Repositorio<T extends Identificable>
```

Esto mejora:

- reutilización
- mantenimiento
- claridad
- escalabilidad

Si mañana se agrega `Usuario implements Identificable`, también podría usar el mismo repositorio.

Relación con Spring Boot

Este concepto prepara el camino para entender estructuras reales como:

```
JpaRepository<Articulo, Integer>
```

En Spring Boot, los repositorios también usan generics.

Por eso este ejemplo es muy útil como transición desde Java consola hacia backend profesional.

Cierre del Manual 4

Este proyecto enseña a generalizar lógica repetida.

Los conceptos clave son:

- generico `T`
- restricción `T extends Identificable`

- interfaz `Identificable`
 - repositorio genérico
 - reutilización de código
 - protección de la lista interna
 - separación inicial entre lógica de almacenamiento y lógica de menú
-

CRUD 5 — Menús Separados

Proyecto CRUD con separación de menús

Tema central

Este proyecto reorganiza el código separando los menús en clases distintas.

Aparecen nuevos archivos:

```
src/com/techlab/articulo/menu/Menu.java
src/com/techlab/articulo/menu/MenuArticulos.java
src/com/techlab/articulo/menu/MenuCategorias.java
```

El objetivo es reducir el tamaño de `App` y mejorar la separación de responsabilidades.

Problema antes de separar menús

En las clases anteriores, `App` hacía demasiadas cosas:

- mostraba el menú principal
- leía datos por consola
- administraba artículos
- administraba categorías
- validaba datos
- buscaba objetos
- modificaba listas

Esto funciona, pero no escala bien.

Cuando una clase hace demasiadas cosas, se vuelve difícil de leer, modificar y mantener.

Nueva organización

Ahora `App` queda más simple.

Su responsabilidad principal es:

- crear el `Scanner`
- crear los repositorios
- crear los menús
- mostrar el menú principal
- derivar la ejecución al menú correspondiente

La lógica específica de artículos se mueve a:

`MenuArticulos`

La lógica específica de categorías se mueve a:

`MenuCategorias`

La lógica común de lectura se mueve a:

`Menu`

Clase abstracta `Menu`

`Menu` es una clase abstracta.

Representa la idea general de un menú.

No queremos crear un menú genérico sin comportamiento concreto.

Queremos crear menús específicos:

- menú de artículos
- menú de categorías

Por eso tiene sentido que `Menu` sea abstracta.

Herencia aplicada a menús

Las clases hijas son:

```
public class MenuArticulos extends Menu
```

Y:

```
public class MenuCategorias extends Menu
```

Esto significa:

- `MenuArticulos` es un `Menu`
- `MenuCategorias` es un `Menu`

Ambas clases comparten lógica común de lectura por consola heredada desde `Menu`.

Métodos abstractos en `Menu`

La clase `Menu` puede definir métodos abstractos como:

```
public abstract void mostrarMenu();  
public abstract void ejecutar();
```

La clase base sabe que todo menú debe poder mostrarse y ejecutarse.

Pero no sabe qué opciones tendrá cada menú.

Por eso cada clase hija implementa su propia versión.

Implementación en `MenuArticulos`

`MenuArticulos` muestra opciones relacionadas con artículos:

```
System.out.println("1 - Ingresar artículo");  
System.out.println("2 - Consultar artículos");  
System.out.println("3 - Consultar un artículo");  
System.out.println("4 - Modificar un artículo");  
System.out.println("5 - Eliminar un artículo");  
System.out.println("0 - Volver");
```

Y ejecuta métodos como:

```
ingresarArticulo();  
listarArticulos();  
consultarArticulo();  
modificarArticulo();  
eliminarArticulo();
```

Implementación en MenuCategorias

MenuCategorias muestra opciones relacionadas con categorías:

```
System.out.println("1 - Ingresar categoría");
System.out.println("2 - Consultar categorías");
System.out.println("3 - Consultar una categoría");
System.out.println("4 - Modificar una categoría");
System.out.println("5 - Eliminar una categoría");
System.out.println("0 - Volver");
```

Y ejecuta métodos como:

```
ingresarCategoria();
listarCategorias();
consultarCategoria();
modificarCategoria();
eliminarCategoria();
```

Reutilización de lectura de datos

La clase Menu concentra métodos comunes como:

```
leerEntero(...)
leerEnteroNoNegativo(...)
leerDoubleNoNegativo(...)
leerTextoNoVacio(...)
```

Esto evita repetir esa lógica en cada menú.

Antes, si había que cambiar la forma de leer un número, había que buscar el código en App.

Ahora esa lógica está centralizada en Menu.

Constructores y dependencias

Los menús necesitan acceder a repositorios.

Por ejemplo, MenuArticulos necesita:

- repositorio de artículos
- repositorio de categorías

Porque para crear un artículo necesita elegir una categoría existente.

Entonces recibe esos objetos por constructor.

Esto es una forma inicial de inyección de dependencias manual.

Separación de responsabilidades

La separación de responsabilidades significa que cada clase debe tener un motivo claro para existir.

En esta etapa:

- `App` coordina el inicio del programa
- `Menu` contiene comportamiento común de menús
- `MenuArticulos` administra artículos
- `MenuCategorias` administra categorías
- `Repositorio` administra almacenamiento en memoria
- `Articulo` y `Categoria` representan datos del dominio

El código queda más ordenado.

Cierre del Manual 5

Este proyecto enseña una mejora importante de diseño.

Los conceptos clave son:

- separación de responsabilidades
 - clase abstracta `Menu`
 - herencia aplicada a menús
 - métodos abstractos para comportamiento obligatorio
 - reutilización de métodos comunes
 - reducción del tamaño de `App`
 - organización del proyecto por paquetes
-

CRUD 6 — Preentrega Completa

Proyecto CRUD completo con validaciones y utilitarios

Tema central

Este proyecto representa una versión más completa y prolija del CRUD.

Mantiene los conceptos anteriores y agrega una mejor organización mediante clases utilitarias.

Aparecen archivos como:

```
src/com/techlab/articulo/utils/Validaciones.java  
src/com/techlab/articulo/utils/Secuencias.java
```

También se consolida el uso de:

- modelos
- interfaces
- repositorio genérico
- menús separados
- validaciones centralizadas
- generación automática de códigos

Estructura general

El proyecto completo queda organizado en paquetes:

```
com.techlab.articulo  
com.techlab.articulo.menu  
com.techlab.articulo.model  
com.techlab.articulo.interfaces  
com.techlab.articulo.repository  
com.techlab.articulo.utils
```

Esta organización es mucho más profesional que tener todo en una sola clase.

Cada paquete agrupa responsabilidades relacionadas.

Paquete `model`

Contiene las clases que representan el dominio del sistema:

```
Articulo  
ArticuloElectronico  
ArticuloAlimenticio  
Categoria
```

Estas clases responden a la pregunta:

```
¿Qué objetos existen en el sistema?
```


Paquete `interfaces`

Contiene contratos:

```
Calculable  
Identificable
```

`Calculable` obliga a calcular precio final.

`Identificable` obliga a exponer un código.

Paquete `repository`

Contiene:

```
Repositorio
```

Su responsabilidad es administrar listas en memoria de forma genérica.

Permite trabajar con:

```
Repositorio<Articulo>  
Repositorio<Categoria>
```

Paquete `menu`

Contiene:

```
Menu  
MenuArticulos  
MenuCategorias
```

Su responsabilidad es manejar interacción con el usuario por consola.

Paquete `utils`

Contiene clases utilitarias:

Validaciones
Secuencias

Estas clases no representan objetos del dominio.

No son artículos ni categorías.

Son clases de apoyo.

Clase utilitaria `Validaciones`

A esta altura del proyecto empieza a aparecer una idea muy importante:

no toda clase representa un objeto del mundo real

`Articulo` representa un artículo.

`Categoria` representa una categoría.

Pero `Validaciones` no representa un objeto real.

No existe “una validación” como entidad del sistema.

La clase existe solamente para agrupar funciones relacionadas.

A este tipo de clase se la suele llamar:

- clase utilitaria
- utility class
- helper class

`Validaciones` se declara así:

```
public final class Validaciones {
```

Se usa `final` para indicar que no debería heredarse.

Como es una clase utilitaria, no se espera crear clases hijas de ella.

Constructor privado en una clase utilitaria

`Validaciones` tiene constructor privado:

```
private Validaciones() {  
}
```

Esto impide hacer:

```
new Validaciones();
```

¿Por qué?

Porque no necesitamos crear objetos de `Validaciones`.

Solo queremos usar sus métodos estáticos.

Métodos estáticos

Un método estático pertenece a la clase, no a un objeto específico.

Ejemplo:

```
public static boolean validarTextoNoVacio(String texto) {  
    return texto != null && !texto.trim().isEmpty();  
}
```

Se usa así:

```
Validaciones.validarTextoNoVacio(nombre)
```

No hace falta crear un objeto.

Validaciones centralizadas

La clase `Validaciones` incluye métodos como:

```
validarTextoNoVacio(...)  
validarLongitudMaxima(...)  
validarNoNegativo(int valor)  
validarNoNegativo(double valor)
```

Esto evita repetir condiciones por todo el programa.

Además, si mañana cambia una regla, se modifica en un solo lugar.

Sobrecarga de métodos

En `Validaciones` aparecen dos métodos con el mismo nombre:

```
public static boolean validarNoNegativo(int valor)
```

Y:

```
public static boolean validarNoNegativo(double valor)
```

Esto se llama sobrecarga.

La sobrecarga permite tener varios métodos con el mismo nombre, siempre que reciban parámetros diferentes.

Java decide cuál usar según el tipo de dato enviado.

Clase utilitaria `Secuencias`

`Secuencias` centraliza la generación de códigos automáticos.

Tiene atributos estáticos:

```
private static int proximoCodigoArticulo = 1;  
private static int proximoCodigoCategoria = 1;
```

Estos atributos pertenecen a la clase, no a un objeto.

Generación automática de códigos

El método:

```
public static int generarCodigoArticulo() {  
    return proximoCodigoArticulo++;  
}
```

Devuelve el próximo código disponible y luego incrementa el contador.

Lo mismo para categorías:

```
public static int generarCodigoCategoria() {  
    return proximoCodigoCategoria++;  
}
```

Esto evita pedirle al usuario que ingrese manualmente el código.

También reduce errores y duplicados.

Operador post-incremento

En esta línea:

```
return proximoCodigoArticulo++;
```

El operador `++` está después de la variable.

Eso significa:

1. primero devuelve el valor actual
2. después incrementa la variable

Si `proximoCodigoArticulo` vale 1, el método devuelve 1 y luego deja la variable en 2.

Validaciones en los menús

En la preentrega, los menús ya no solo leen datos.

También validan mejor.

Por ejemplo:

- que el nombre no esté vacío
- que el precio no sea negativo
- que la garantía no sea negativa
- que no haya artículos duplicados
- que no se eliminen categorías con artículos asociados

Esto acerca el proyecto a una lógica más real.

Duplicados de artículos

En `MenuArticulos` aparece una validación como:

```
existeArticuloDuplicado(...)
```

Esta validación evita cargar dos artículos funcionalmente iguales.

Aunque tengan códigos distintos, desde el punto de vista del usuario podría ser un error cargar el mismo artículo repetido.

Repositorio más completo

El repositorio de esta versión incluye también:

```
cantidad()
```

Este método devuelve la cantidad de elementos almacenados.

```
public int cantidad() {  
    return lista.size();  
}
```

Esto puede servir para reportes, mensajes o validaciones.

Diseño general final

La preentrega combina varios conceptos:

```
P00 básica  
Encapsulamiento  
Herencia  
Polimorfismo  
Clases abstractas  
Métodos abstractos  
Interfaces  
Generics  
Repositorios  
Menús separados  
Clases utilitarias  
Métodos estáticos  
Sobrecarga  
Validaciones  
Secuencias automáticas
```

Esto muestra una evolución clara desde un CRUD simple hacia una estructura más mantenible.

Cierre del Manual 6

La preentrega resuelta completa muestra cómo organizar mejor un proyecto Java de consola.

Los aprendizajes principales son:

- separar el dominio en clases modelo
- usar interfaces para contratos
- usar generics para evitar duplicación
- separar menús por responsabilidad
- centralizar validaciones
- centralizar generación de códigos
- evitar clases gigantes
- preparar la lógica para futuros proyectos más grandes

Este diseño todavía es de consola y memoria, pero ya empieza a parecerse a una arquitectura más profesional.

Muchas ideas de este proyecto se reutilizan luego en aplicaciones con Spring Boot, donde aparecen capas como:

- controller
- service
- repository
- model/entity
- dto

Por eso esta preentrega funciona como puente entre Java POO básico y desarrollo backend más avanzado.